

第一篇：AM-GCN

论文链接: <https://arxiv.org/pdf/2007.02265>

源码链接: <https://github.com/zhumeiqiBUPT/AM-GCN>



任务目标:

1. 通读全文，翻译内容写入该文档
2. 复现代码任务，并整理好复现数据发给我（不同L/C下各数据集的ACC和F1）

摘要

图卷积网络（Graph Convolutional Networks, GCNs）在处理图数据和网络数据的各种分析任务中获得了极大的关注。然而，近期一些研究引起了人们对GCNs是否能够在具有丰富信息的复杂图中最优地融合节点特征（node features）和拓扑结构（topological structures）的关注。在本文中，我们首先进行了实验研究。令人惊讶的是，我们的实验结果清楚表明，当前最先进的GCNs在融合节点特征和拓扑结构方面的能力距离最优效果仍然较远，甚至无法达到令人满意的水平。由于GCNs可能无法自适应地学习拓扑结构与节点特征之间的深层关联信息，这一缺陷可能会严重限制其在某些分类任务中的性能。我们是否能够弥补这一缺陷，并设计一种新型GCNs，使其既能够保留当前最先进GCNs的优势，同时又能够显著增强融合拓扑结构和节点特征的能力？针对这一挑战，我们提出了一种用于半监督分类（semi-supervised classification）的自适应多通道图卷积网络（Adaptive Multi-channel Graph Convolutional Networks, AM-GCN）。其核心思想是同时从节点特征、拓扑结构以及二者的组合信息中提取特定表示（specific embeddings）和公共表示（common embeddings），并利用注意力机制（attention mechanism）学习这些表示的自适应重要性权重。我们在基准数据集上的大量实验结果清楚表明，AM-GCN能够从节点特征和拓扑结构中充分提取最相关的信息，并且能够显著提高分类准确率。

1. 引言

网络数据无处不在，例如社交网络、生物网络以及引用网络。近年来，图卷积网络（Graph Convolutional Networks, GCNs）作为一类用于学习图数据的神经网络，在解决图分析问题方面受到了广泛关注，例如节点分类（node classification）、图分类（graph classification）、链接预测（link prediction）以及推荐（recommendation）等任务。

典型的GCN及其变体通常遵循一种消息传递（message-passing）的方式。其中，一个关键步骤是特征聚合（feature aggregation），即在每一个卷积层中，一个节点会从其拓扑邻居（topological neighbors）中聚合特征信息。通过这种方式，特征信息会沿着网络拓扑结构传播到节点嵌入（node

embedding) 中, 随后, 通过这种方式学习得到的节点嵌入会被用于分类任务。整个过程由节点标签 (node labels) 进行部分监督。GCN 能够取得巨大成功, 部分原因在于 GCN 提供了一种融合拓扑结构信息和节点特征的方法, 用于学习节点嵌入, 并且这种融合过程是在端到端学习框架 (end-to-end learning framework) 下进行监督的。

然而, 近期一些研究揭示了当前最先进的 GCNs (state-of-the-art GCNs) 在融合节点特征和拓扑结构方面存在一定的不足。例如, Li 等人表明, GCNs 实际上会对节点特征执行拉普拉斯平滑 (Laplacian smoothing), 并使整个网络中的节点嵌入逐渐趋于一致。Nt 和 Maehara 以及 Wu 等人证明, 当特征信息沿网络拓扑结构传播时, 拓扑结构会对节点特征起到低通滤波 (low-pass filtering) 的作用。Gao 等人在 GCN 中设计了一个条件随机场 (Conditional Random Field, CRF) 层, 以显式地保持节点之间的连通性。

GCNs 到底从拓扑结构和节点特征中学习并融合了哪些信息? 这是一个基础性问题, 因为 GCNs 经常被用作一种端到端学习框架。对于这一问题, 一个充分理解的、基于理论依据的回答, 可以帮助我们以更加系统的方式理解 GCNs 的能力和局限性。这也直接促使了我们的研究。

作为本研究的第一个贡献, 我们通过实验评估了 GCNs 在融合拓扑结构和节点特征方面的能力。令人惊讶的是, 我们的实验结果清楚表明, GCNs 在融合网络拓扑结构和节点特征方面的能力距离最优效果仍然存在明显差距, 甚至无法达到令人满意的水平。即使在一些简单情况下——节点特征/拓扑结构与节点标签之间的相关性非常明显, GCNs 仍然无法充分融合节点特征和拓扑结构, 以提取其中最相关的信息 (如第 2 节所示)。由于 GCNs 可能无法自适应地学习拓扑结构与节点特征之间的某些关联信息, 这一不足可能会严重限制 GCNs 在某些分类任务中的能力。

一旦确定当前最先进的 GCNs 在信息融合方面存在不足, 一个自然的问题是: “我们是否能够弥补这一缺陷, 并设计一种新型的 GCNs, 使其既能够保留当前最先进 GCNs 的优势, 同时又能够显著增强融合拓扑结构和节点特征的能力?”

一个具有良好融合能力的 GCNs 应该能够为分类任务充分提取并融合最相关的信息。然而, 在实际情况下, 最大的障碍之一是网络数据与分类任务之间的相关性通常非常复杂且难以确定。分类结果可能与以下任一因素相关: 拓扑结构、节点特征或者二者的组合信息。本文针对这一挑战, 提出了一种用于半监督分类的自适应多通道图卷积网络 (Adaptive Multi-channel Graph Convolutional Networks, AM-GCN)。其核心思想是同时基于节点特征、拓扑结构以及二者的组合信息学习节点嵌入。其基本依据在于: 由特征信息体现出的相似性与由拓扑结构推断出的相似性彼此互补, 并且二者可以通过自适应融合的方式结合起来, 从而为分类任务提取更深层次的关联信息。

从技术角度来看, 为了充分利用特征空间 (feature space) 中的信息, 我们根据节点特征生成 k 近邻图 (k-nearest neighbor graph), 并将其作为特征结构图 (feature structural graph)。利用特征图 (feature graph) 和拓扑图 (topology graph), 我们分别在拓扑空间 (topology space) 和特征空间 (feature space) 中传播节点特征, 从而通过两个特定的卷积模块 (specific convolution modules) 提取这两个空间中的两个特定表示 (specific embeddings)。考虑到两个空间之间存在的共同特征, 我们设计了一个具有参数共享策略 (parameter sharing strategy) 的公共卷积模块 (common convolution module), 用于提取二者共享的共同表示 (common embedding)。进一步地, 我们利用注意力机制 (attention mechanism) 自动学习不同表示的重要性权重, 从而实现自适应融合 (adaptive fusion)。通过这种方式, 节点标签能够监督学习过程, 使模型能够自适应地调

整权重，从而提取与分类任务最相关的信息。此外，我们设计了一致性约束（consistency constraint）和差异性约束（disparity constraint），以确保所学习得到的表示具有一致性和差异性。我们总结本文的主要贡献如下：

- 我们通过实验评估了 GCN 在融合拓扑结构和节点特征方面的能力，并发现了 GCN 存在的不足。进一步地，我们研究了一个重要问题，即如何显著增强 GCN 在分类任务中的融合能力。
- 我们提出了一种新型的自适应多通道 GCN 框架——AM-GCN，该框架分别在拓扑空间（topology space）和特征空间（feature space）上执行图卷积操作。结合注意力机制（attention mechanism），不同的信息能够得到充分融合。
- 我们在一系列基准数据集（benchmark datasets）上的大量实验结果清楚表明，AM-GCN 的性能优于当前最先进的 GCN 方法，并且能够在具有挑战性的分类任务中，从节点特征和拓扑结构中有效提取最相关的信息。

本文其余部分的组织结构如下：第 2 节中，我们通过实验研究 GCN 融合节点特征和拓扑结构的能力；第 3 节中，我们提出 AM-GCN 模型；第 4 节中，我们报告实验结果，并在第 5 节中回顾相关工作；最后，我们在第 6 节中总结全文。

2. GCNs 的融合能力：一项实验研究

在本节中，我们通过两个简单但直观的案例，检验当前最先进的 GCNs 是否能够在图中自适应地从节点特征和拓扑结构中学习信息，并针对分类任务充分地融合这些信息。主要思想是：我们将分别建立节点标签与网络拓扑结构、节点特征之间的高度相关性，然后在这两个简单案例上测试 GCN 的性能。一个具有良好融合能力的 GCN 应该能够在节点标签的监督下，自适应地提取与分类任务相关的信息，并取得良好的分类效果。然而，如果与基线方法（baselines）相比，GCN 的性能出现明显下降，这将说明：即使节点特征或拓扑结构与节点标签之间存在高度相关性，GCN 仍然无法自适应地从节点特征和拓扑结构中提取有效信息。

2.1 案例1：随机拓扑结构与相关节点特征

我们生成了一个包含 900 个节点的随机网络，其中任意两个节点之间建立边的概率为 0.03。每个节点具有一个 50 维的特征向量。为了生成节点特征，我们随机为 900 个节点分配 3 个标签。对于具有相同标签的节点，我们使用同一个高斯分布（Gaussian distribution）来生成节点特征。这三个类别节点对应的高斯分布具有相同的协方差矩阵（covariance matrix），但三个分布的中心彼此相距较远。在该数据集中，节点标签与节点特征之间具有高度相关性，但与拓扑结构之间并不存在相关性。

我们使用 GCN 对该网络进行训练。对于每一个类别，我们随机选择 20 个节点作为训练节点，另外选择 200 个节点作为测试节点。我们仔细调整超参数，以获得最佳性能，并避免过度平滑问题。此外，我们仅基于节点特征应用 MLP 模型进行实验。GCN 和 MLP 的分类准确率分别为 75.2% 和 100%。

实验结果符合预期。由于节点特征与节点标签高度相关，MLP 展现出了优异的性能。GCN 同时从节点特征和拓扑结构中提取信息，但无法自适应地融合这两类信息，以避免拓扑结构带来的干扰。因此，GCN 无法达到 MLP 的高性能表现。

2.2 案例2：相关拓扑结构与随机节点特征

我们生成了另一个包含 900 个节点的网络。这一次，每个节点的特征由 50 维向量组成，并且这些特征是随机生成的。

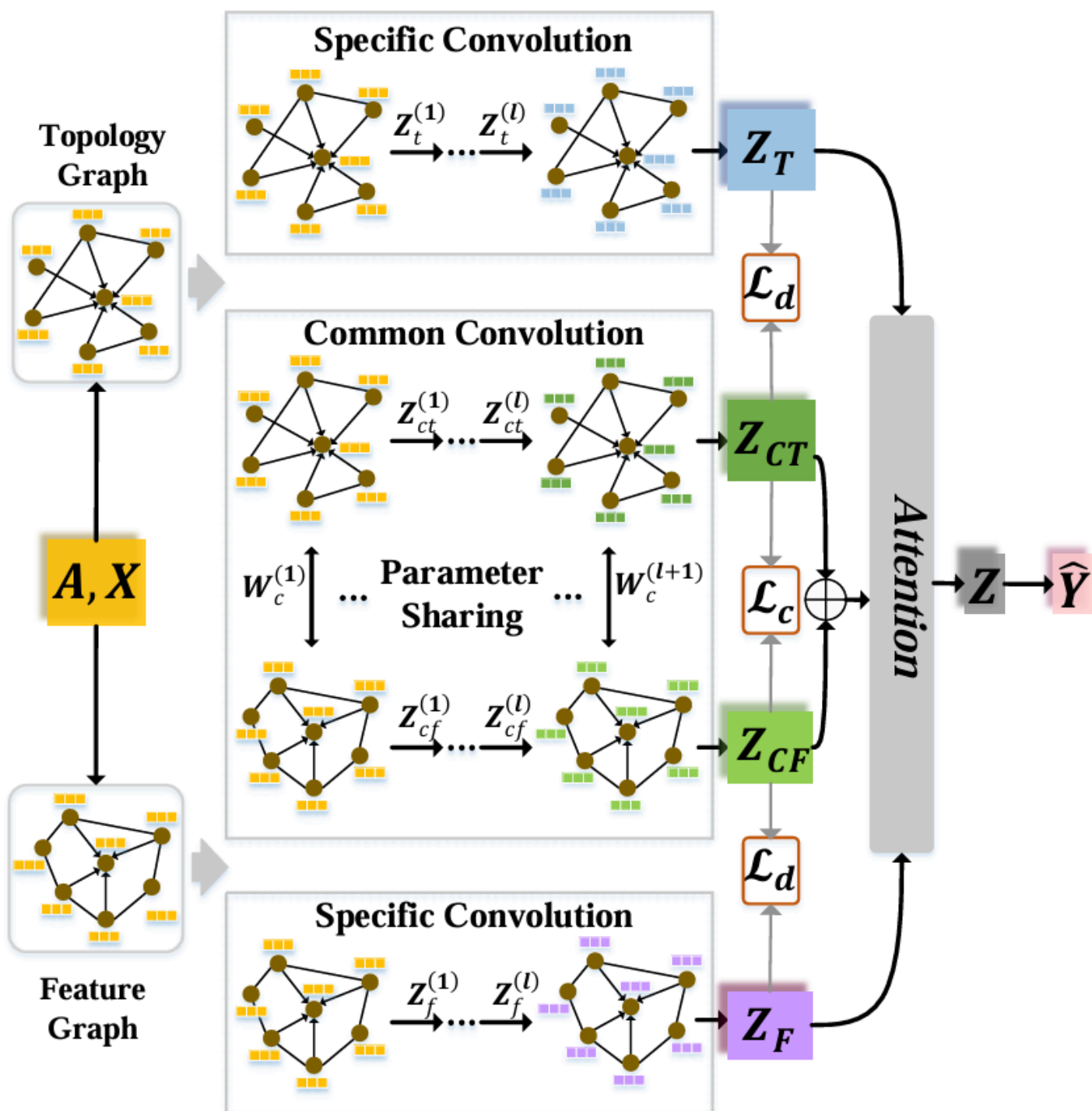


图1：AM-GCN模型框架。节点特征 X 用于构建特征图（feature graph）。AM-GCN 包含两个特定卷积模块（specific convolution modules），一个公共卷积模块（common convolution module），一个注意力机制（attention mechanism）。

对于拓扑结构，我们采用随机块模型（Stochastic Block Model, SBM）将节点划分为 3 个社区（分别为节点 0-299、300-599、600-899）。在每个社区内部，任意两个节点之间建立边的概率设置为 0.03；而不同社区节点之间建立边的概率设置为 0.0015。在该数据集中，节点标签由社区决定，即属于同一社区的节点具有相同的标签。

我们再次将 GCN 应用于该网络。此外，我们还将 DeepWalk 应用于该网络的拓扑结构，也就是说，DeepWalk 忽略了节点特征信息。GCN 和 DeepWalk 的分类准确率分别为 87% 和 100%。DeepWalk 表现良好的原因在于，它能够充分建模网络拓扑结构信息。GCN 同时从节点特征和拓扑结构中提取信息，但无法自适应地融合二者，从而避免节点特征带来的干扰。因此，GCN 无法达到 DeepWalk 的高性能表现。

总结

这些案例表明，当前 GCN 的融合机制距离最优效果仍然存在明显差距，甚至无法达到令人满意的水平。即使节点标签与网络拓扑结构或节点特征之间具有非常高的相关性，当前 GCN 仍然无法充分利用节点标签提供的监督信息，自适应地提取最相关的信息。然而，在现实情况下，问题会更加复杂，因为我们通常难以判断：到底是拓扑结构，还是节点特征，与最终任务具有更强的相关性。这促使我们重新思考当前 GCN 的信息融合机制。

3. AM-GCN：提出的模型

问题设置：我们研究的是带属性图 $G = (A, X)$ 上的半监督节点分类问题（semi-supervised node classification）。其中 $A \in \mathbb{R}^{n \times n}$ 是一个具有 n 个节点的对称邻接矩阵（symmetric adjacency matrix）， $X \in \mathbb{R}^{n \times d}$ 是节点特征矩阵（node feature matrix），其中 d 表示节点特征的维度。具体来说， $A_{ij} = 1$ 表示节点 i 和节点 j 之间存在一条边；否则， $A_{ij} = 0$ 。我们假设每个节点属于 C 个类别中的某一个类别。

AM-GCN 的整体框架如图1所示。其核心思想是，AM-GCN 允许节点特征不仅在拓扑空间（topology space）中传播，也能够特征空间（feature space）中传播，并且需要从这两个空间中提取与节点标签最相关的信息。为此，我们基于节点特征 X 构建了一个特征图。随后，通过两个特定的卷积模块，使 X 能够分别在特征图和拓扑图上传播，从而学习得到两个特定的嵌入表示 Z_F 和 Z_T 。进一步地，考虑到这两个空间中的信息具有共同特征，我们设计了一个采用参数共享策略（parameter sharing strategy）的公共卷积模块，用于学习公共嵌入表示 Z_{CF} 和 Z_{CT} 。同时，引入一致性约束 L_c ，以增强 Z_{CF} 和 Z_{CT} 的“公共”属性。此外，差异性约束 L_d 用于确保 Z_F 与 Z_{CF} 之间以及 Z_T 与 Z_{CT} 之间的独立性。考虑到节点标签可能与拓扑信息、特征信息或者两者都存在相关性，AM-GCN 利用注意力机制（attention mechanism），通过学习得到的权重自适应地融合这些嵌入表示，从而提取与节点标签最相关的信息 Z ，用于最终的分类任务。

3.1 特定卷积模块

首先，为了捕获节点在特征空间中的潜在结构，我们基于节点特征矩阵 X 构建了一个 k 近邻（k-nearest neighbor, kNN）图 $G_f = (A_f, X)$ ，其中， A_f 是 kNN 图的邻接矩阵（adjacency matrix）。具体来说，我们首先计算 n 个节点之间的相似度矩阵（similarity matrix） $S \in \mathbb{R}^{n \times n}$ 。实际上，获取 S 的方法有很多种。这里列举其中两种常用的方法，其中 x_i 和 x_j 分别表示节点 i 和节点 j 的特征向量：

1) 余弦相似度（Cosine Similarity）：它利用两个向量夹角的余弦值来衡量两个向量之间的相似程度。

$$S_{ij} = \frac{x_i \cdot x_j}{|x_i||x_j|} \quad (1)$$

2) **热核 (Heat Kernel)**：相似度通过公式 (2) 计算，其中 t 是热传导方程中的时间参数 (time parameter)，本文设置 $t = 2$

$$S_{ij} = e^{-\frac{|x_i - x_j|^2}{t}} \quad (2)$$

在本文中，我们统一选择余弦相似度 (Cosine Similarity) 来获得相似度矩阵 S 。然后，对于每个节点，我们选择与其最相似的 k 个节点对，并设置边连接，最终得到邻接矩阵 A_f 。

随后，将特征空间中的输入图 (A_f, X) 输入后，第 I 层的输出 $Z_f^{(l)}$ 可以表示为：

$$Z_f^{(l)} = \text{ReLU}(\tilde{D}_f^{-\frac{1}{2}} \tilde{A}_f \tilde{D}_f^{-\frac{1}{2}} Z_f^{(l-1)} W_f^{(l)}) \quad (3)$$

其中 $W_f^{(l)}$ 是 GCN 第 I 层的权重矩阵 (weight matrix)，ReLU 是 ReLU 激活函数 (ReLU activation function) 和初始状态 $Z_f^{(0)} = X$ 。具体来说，我们有 $\tilde{A}_f = A_f + I_f$ ， $\tilde{D}_f$ 是 $\tilde{A}_f$ 的对角度矩阵 (diagonal degree matrix)。我们将最后一层的输出嵌入表示记为 Z_F 。通过这种方式，我们可以学习到能够捕获特征空间中特定信息 (specific information in feature space) 的节点嵌入表示 Z_F 。

对于拓扑空间 (topology space)，我们使用原始输入图 $G_t = (A_t, X_t)$ ，其中， $A_t = A$ ， $X_t = X$

然后，基于拓扑图学习得到的输出嵌入表示 Z_T ，可以采用与特征空间中相同的方式进行计算。因此，可以提取出编码在拓扑空间 (topology space) 中的特定信息。

3.2 公共卷积模块

实际上，特征空间 (feature space) 和拓扑空间 (topology space) 并不是完全无关的。通常情况下，节点分类任务可能与以下信息相关：特征空间中的信息、拓扑空间中的信息或者两个空间中的共同信息。而这些信息在事先是难以确定的。因此，我们不仅需要提取这两个空间中的节点特定嵌入 (node specific embedding)，还需要提取两个空间之间共享的公共信息 (common information)。这样，对于任务而言，确定哪一部分信息与节点分类最相关会更加灵活。为了解决这一问题，我们设计了一个采用参数共享策略 (parameter sharing strategy) 的 Common-GCN，用于获取两个空间中共享的嵌入表示。

首先，我们利用 Common-GCN 从拓扑图 (A_t, X) 中提取节点嵌入 $Z_{ct}^{(l)}$ ，计算如下：

$$Z_{ct}^{(l)} = \text{ReLU}(\tilde{D}_t^{-\frac{1}{2}} \tilde{A}_t \tilde{D}_t^{-\frac{1}{2}} Z_{ct}^{(l-1)} W_c^{(l)}) \quad (4)$$

其中 $W_c^{(l)}$ 是 Common-GCN 第 I 层的权重矩阵， $Z_{ct}^{(l-1)}$ 是第 $(l-1)$ 层的节点嵌入初始 $Z_{ct}^{(0)} = X$ 。当利用 Common-GCN 从特征图 (A_f, X) 中学习节点嵌入时，为了提取共享信息，我们在 Common-

GCN 的每一层中共享相同的权重矩阵 $W_c^{(l)}$ ，计算如下：

$$Z_{cf}^{(l)} = ReLU(\tilde{D}_f^{-\frac{1}{2}} \tilde{A}_f \tilde{D}_f^{-\frac{1}{2}} Z_{cf}^{(l-1)} W_c^{(l)}) \quad (5)$$

其中 $Z_{cf}^{(l)}$ 是第 l 层输出的嵌入表示初始 $Z_{cf}^{(0)} = X$ 共享的权重矩阵能够从两个空间中筛选出共享特征 (shared characteristics)。根据不同的输入图，我们可以得到两个输出嵌入 Z_{CT} 和 Z_{CF} 两个空间的公共嵌入表示为：

$$Z_C = \frac{Z_{CT} + Z_{CF}}{2} \quad (6)$$

3.3 注意力机制

现在，我们已经获得两个特定嵌入表示 Z_T 和 Z_F ，一个公共嵌入表示 Z_C 。考虑到节点标签可能与其中某一个嵌入相关，或者与多个嵌入的组合相关，我们使用注意力机制 $att(Z_T, Z_C, Z_F)$ 来学习它们对应的重要性权重 $(\alpha_t, \alpha_c, \alpha_f)$ 计算如下：

$$(\alpha_t, \alpha_c, \alpha_f) = att(Z_T, Z_C, Z_F) \quad (7)$$

其中， $\alpha_t, \alpha_c, \alpha_f \in \mathbb{R}^{n \times 1}$ 分别表示具有嵌入 Z_T, Z_C, Z_F 的 n 个节点对应的注意力值。

这里我们关注节点 i 。其在 Z_T 中的嵌入表示为 $z_T^i \in \mathbb{R}^{1 \times h}$ (即 Z_T 的第 i 行)。首先，我们通过非线性变换对嵌入进行转换，然后利用一个共享的注意力向量 $q \in \mathbb{R}^{h' \times 1}$ 计算注意力值：

$$\omega_T^i = q^T \cdot \tanh(W \cdot (z_T^i)^T + b) \quad (8)$$

其中， $W \in \mathbb{R}^{h' \times h}$ 是权重矩阵， $b \in \mathbb{R}^{h' \times 1}$ 是偏置向量。类似地，我们可以分别得到节点 i 在嵌入矩阵 Z_C 和 Z_F 中对应的注意力值 ω_C^i 和 ω_F^i 。然后，我们利用 softmax 函数对注意力值： $\omega_T^i, \omega_C^i, \omega_F^i$ 进行归一化，得到最终权重：

$$\alpha_T^i = softmax(\omega_T^i) = \frac{\exp(\omega_T^i)}{\exp(\omega_T^i) + \exp(\omega_C^i) + \exp(\omega_F^i)} \quad (9)$$

较大的 α_T^i 表明对应的嵌入表示更加重要。类似地 $\alpha_C^i = softmax(\omega_C^i)$ 以及 $\alpha_F^i = softmax(\omega_F^i)$ 对于全部 n 个节点，我们得到学习到的权重 $\alpha_t = [\alpha_T^i], \alpha_c = [\alpha_C^i], \alpha_f = [\alpha_F^i] \in \mathbb{R}^{n \times 1}$ 并定义 $\alpha_T = diag(\alpha_t), \alpha_C = diag(\alpha_c), \alpha_F = diag(\alpha_f)$ ，然后，将这三个嵌入表示进行融合，得到最终嵌入表示 Z ：

$$Z = \alpha_T \cdot Z_T + \alpha_C \cdot Z_C + \alpha_F \cdot Z_F \quad (10)$$

3.4 目标函数

3.4.1 一致性约束

对于 Common-GCN 的两个输出嵌入 Z_{CT} 和 Z_{CF} 尽管 Common-GCN 使用了共享权重矩阵，但为了进一步增强它们之间的公共特性，我们设计了一种一致性约束。

首先，我们使用 L2 归一化（L2-normalization）对嵌入矩阵进行归一化： Z_{CTnor}, Z_{CFnor} 。然后，利用归一化后的两个矩阵计算 n 个节点之间的相似度：

$$\begin{aligned} S_T &= Z_{CTnor} \cdot (Z_{CTnor})^T \\ S_F &= Z_{CFnor} \cdot (Z_{CFnor})^T \end{aligned} \quad (11)$$

一致性意味着两个相似度矩阵应该保持相似，因此得到如下约束：

$$L_c = ||S_T - S_F||_F^2 \quad (12)$$

3.4.2 差异性约束

由于嵌入表示 Z_T 和 Z_{CT} 都是从同一个图 $G_t = (A_t, X_t)$ 中学习得到的。为了确保它们能够捕获不同的信息，我们采用 Hilbert-Schmidt 独立性准则（Hilbert-Schmidt Independence Criterion, HSIC），这是一种简单但有效的独立性度量方法，用于增强两个嵌入表示之间的差异性。由于 HSIC 具有简单性和良好的理论性质，它已经被应用于多个机器学习任务中 [10,19]。形式上， Z_T 和 Z_{CT} 的 HSIC 约束定义为：

$$HSIC(Z_T, Z_{CT})(n-1)^{-2}tr(RK_T RK_{CT}) \quad (13)$$

其中， K_T 和 K_{CT} 是 Gram 矩阵； $k_{T,ij} = k_T(z_T^i, z_T^j)$ $k_{CT,ij} = k_{CT}(z_{CT}^i, z_{CT}^j)$ ，并且 $R = I - \frac{1}{n}ee^T$ ，其中 I 是单位矩阵； e 是全1列向量。在本文实现中，我们使用内积核函数（inner product kernel function）计算 K_T 和 K_{CT}

类似地，由于嵌入表示 Z_F 和 Z_{CF} 同样来自同一个图 A_f, X 。因此，它们之间的差异性也应该通过 HSIC 进行增强：

$$HSIC(Z_F, Z_{CF})(n-1)^{-2}tr(RK_F RK_{CF}) \quad (14)$$

因此，我们设置差异性约束 L_d ，其中：

$$L_d = HSIC(Z_T, Z_{CT}) + HSIC(Z_F, Z_{CF}) \quad (15)$$

Table 1: The statistics of the datasets

Dataset	Nodes	Edges	Classes	Features	Training	Test
Citeseer	3327	4732	6	3703	120/240/360	1000
UAI2010	3067	28311	19	4973	380/760/1140	1000
ACM	3025	13128	3	1870	60/120/180	1000
BlogCatalog	5196	171743	6	8189	120/240/360	1000
Flickr	7575	239738	9	12047	180/360/540	1000
CoraFull	19793	65311	70	8710	1400/2800/4200	1000

3.4.3 优化目标

我们使用公式 (10) 中的输出嵌入 Z ，通过线性变换和 softmax 函数进行半监督多类别分类。将 n 个节点的类别预测结果表示为： $\hat{Y} = [\hat{y}_{ic}] \in R^{n \times C}$ ，其中， $\hat{y}_{ic}$ 表示节点 i 属于类别 c 的概率。然后， $\hat{Y}$ 可以通过以下方式计算：

$$\hat{Y} = \text{softmax}(W \cdot Z + b) \quad (16)$$

其中 $\text{softmax}(x) = \frac{\exp(x)}{\sum_{c=1}^C \exp(x_c)}$ 实际上是一个跨所有类别的归一化器。

假设训练集为 L ，对于每一个 $l \in L$ ，真实标签为 Y_l ，预测标签为 $\hat{Y}_l$ 。那么，针对所有训练节点的节点分类交叉熵损失可以表示为 L_t ：

$$L_t = - \sum_{l \in L} \sum_{i=1}^C Y_{li} \ln \hat{Y}_{li} \quad (17)$$

结合节点分类任务和约束条件，我们得到整体目标函数：

$$L = L_t + \gamma L_c + \beta L_d \quad (18)$$

其中， γ 和 β 是一致性约束项（consistency constraint）和差异性约束项（disparity constraint）的参数。在标记数据的指导下，我们可以通过反向传播（back propagation）优化所提出的模型，并学习用于分类的节点嵌入表示。

4. 实验

4.1 实验设置

数据集

我们提出的 AM-GCN 模型在六个真实世界数据集上进行了评估，这些数据集总结于表 1 中。此外，为了保证实验的可复现性，我们在补充材料中提供了所有数据集的网站链接。

- **Citeseer**: Citeseer 是一个研究论文引用网络，其中节点表示论文，边表示论文之间的引用关系。节点属性采用论文的词袋模型 (bag-of-words) 表示，并且所有节点被划分为六个研究领域。
- **UAI2010**: 我们使用该数据集进行实验。该数据集包含 3067 个节点和 28311 条边，并且已经在文献 [28] 中用于图卷积网络 (Graph Convolutional Networks) 进行社区检测任务的测试。
- **ACM**: 该网络从 ACM 数据集中提取得到，其中节点表示论文。如果两篇论文具有相同的作者，则它们之间存在一条边。所有论文被划分为 3 个类别: Database (数据库), Wireless Communication (无线通信), Data Mining (数据挖掘)。节点特征采用论文关键词的词袋模型 (bag-of-words) 表示。
- **BlogCatalog**: 这是一个来自 BlogCatalog 网站的社交网络，其中节点表示博客用户，边表示用户之间的社交关系。节点属性由用户个人资料中的关键词构建得到。标签表示由作者提供的主题类别，并且所有节点被划分为 6 个类别。
- **Flickr**: Flickr 是一个图片和视频托管网站，用户通过照片分享进行互动。它是一个社交网络，其中节点表示用户，边表示用户之间的关系。根据用户的兴趣群组，所有节点被划分为 9 个类别。
- **CoraFull [2]**: 这是著名引用网络 Cora 数据集的更大版本，其中节点表示论文，边表示论文之间的引用关系。节点标签根据论文主题进行划分。

基准方法

我们将 AM-GCN 与两类当前最先进的方法 (state-of-the-art methods) 进行比较，包括两种网络嵌入算法 (network embedding algorithms) 和六种基于图神经网络 (Graph Neural Network, GNN) 的方法。此外，为了保证实验的可复现性，我们在补充材料 (supplement) 中提供了所有代码网站链接。

- **DeepWalk**: DeepWalk 是一种网络嵌入方法，它利用随机游走 (random walk) 获取节点的上下文信息，并使用 Skip-Gram 算法学习网络表示。
- **LINE**: LINE 是一种大规模网络嵌入方法，它分别保留网络的一阶邻近性 (first-order proximity) 和二阶邻近性 (second-order proximity)。在实验中，我们使用 LINE (1st+2nd)，即同时结合一阶和二阶邻近性。
- **Chebyshev**: Chebyshev 是一种基于图卷积网络 (GCN) 的方法，它利用 Chebyshev 滤波器 (Chebyshev filters) 进行图卷积操作。
- **GCN**: GCN 是一种半监督图卷积网络模型，通过聚合邻居节点的信息来学习节点表示。
- **kNN-GCN**: 为了进行比较，我们不使用传统的拓扑图 (topology graph)，而是利用节点特征矩阵计算得到的稀疏 k 近邻图 (sparse k-nearest neighbor graph) 作为 GCN 的输入图，并将该方法表示为 kNN-GCN。
- **GAT**: GAT 是一种使用注意力机制 (attention mechanism) 来聚合节点特征的图神经网络模型。

- **DEMO-Net:**

DEMO-Net 是一种用于节点分类的、考虑节点度数特征 (degree-specific) 的图神经网络模型。

- **MixHop:** MixHop 是一种基于 GCN 的方法，它在一个图卷积层 (graph convolution layer) 中融合不同阶邻居 (higher-order neighbors) 的特征表示。

参数设置

为了更加全面地评估我们的模型，我们为训练集选择了三种标签比例 (label rates)，即每个类别分别选取 20、40、60 个有标签节点作为训练数据，并选择 1000 个节点作为测试集。所有基准方法 (baselines) 均采用其论文中推荐的参数进行初始化，同时我们进一步仔细调整参数，以获得最佳性能。对于我们的模型，我们同时训练三个两层 GCN 网络 (2-layer GCNs)，它们具有相同的隐藏层维度 (n_{hid1}) 和相同的输出维度 (n_{hid2})。其中 $n_{hid1} \in \{512, 768\}$, $n_{hid2} \in \{32, 128, 256\}$ 。我们采用 Adam 优化器 (Adam optimizer)，学习率设置范围为 $0.0001 \sim 0.0005$ 此外，dropout 比例设置为 0.5，权重衰减 (weight decay) 设置为 $\{5e-3, 5e-4\}$ ，对于 k 近邻图 (k-nearest neighbor graph)，参数 k 的取值范围为 $k \in \{2, \dots, 10\}$ ，一致性约束 (consistency constraint) 和差异性约束 (disparity constraint) 的系数分别从以下集合中搜索一致性约束系数 $\{0.01, 0.001, 0.0001\}$, 差异性约束系 $\{1e-10, 5e-9, 1e-9, 5e-8, 1e-8\}$ 对于所有方法，我们在相同的数据划分下运行 5 次实验，并报告平均实验结果。我们使用 准确率 (Accuracy, ACC) 和 宏平均 F1 分数 (macro F1-score, F1) 作为模型性能评价指标。为了保证实验的可复现性，我们在补充材料 (Section A.3) 中提供了具体的参数取值。

Table 2: Node classification results(%). (Bold: best; Underline: runner-up.)

Datasets	Metrics	L/C	DeepWalk	LINE	Chebyshev	GCN	kNN-GCN	GAT	DEMO-Net	MixHop	AM-GCN
Citeseer	ACC	20	43.47	32.71	69.80	70.30	61.35	<u>72.50</u>	69.50	71.40	73.10
		40	45.15	33.32	71.64	<u>73.10</u>	61.54	73.04	70.44	71.48	74.70
		60	48.86	35.39	73.26	74.48	62.38	<u>74.76</u>	71.86	72.16	75.56
	F1	20	38.09	31.75	65.92	67.50	58.86	<u>68.14</u>	67.84	66.96	68.42
		40	43.18	32.42	68.31	<u>69.70</u>	59.33	69.58	66.97	67.40	69.81
		60	48.01	34.37	70.31	<u>71.24</u>	60.07	71.60	68.22	69.31	70.92
UAI2010	ACC	20	42.02	43.47	50.02	49.88	<u>66.06</u>	56.92	23.45	61.56	70.10
		40	51.26	45.37	58.18	51.80	<u>68.74</u>	63.74	30.29	65.05	73.14
		60	54.37	51.05	59.82	54.40	<u>71.64</u>	68.44	34.11	67.66	74.40
	F1	20	32.93	37.01	33.65	32.86	<u>52.43</u>	39.61	16.82	49.19	55.61
		40	46.01	39.62	38.80	33.80	<u>54.45</u>	45.08	26.36	53.86	64.88
		60	44.43	43.76	40.60	34.12	54.78	48.97	29.05	<u>56.31</u>	65.99
ACM	ACC	20	62.69	41.28	75.24	<u>87.80</u>	78.52	87.36	84.48	81.08	90.40
		40	63.00	45.83	81.64	<u>89.06</u>	81.66	88.60	85.70	82.34	90.76
		60	67.03	50.41	85.43	<u>90.54</u>	82.00	90.40	86.55	83.09	91.42
	F1	20	62.11	40.12	74.86	<u>87.82</u>	78.14	87.44	84.16	81.40	90.43
		40	61.88	45.79	81.26	<u>89.00</u>	81.53	88.55	84.83	81.13	90.66
		60	66.99	49.92	85.26	<u>90.49</u>	81.95	90.39	84.05	82.24	91.36
BlogCatalog	ACC	20	38.67	58.75	38.08	69.84	<u>75.49</u>	64.08	54.19	65.46	81.98
		40	50.80	61.12	56.28	71.28	<u>80.84</u>	67.40	63.47	71.66	84.94
		60	55.02	64.53	70.06	72.66	<u>82.46</u>	69.95	76.81	77.44	87.30
	F1	20	34.96	57.75	33.39	68.73	<u>72.53</u>	63.38	52.79	64.89	81.36
		40	48.61	60.72	53.86	70.71	<u>80.16</u>	66.39	63.09	70.84	84.32
		60	53.56	63.81	68.37	71.80	<u>81.90</u>	69.08	76.73	76.38	86.94
Flickr	ACC	20	24.33	33.25	23.26	41.42	<u>69.28</u>	38.52	34.89	39.56	75.26
		40	28.79	37.67	35.10	45.48	<u>75.08</u>	38.44	46.57	55.19	80.06
		60	30.10	38.54	41.70	47.96	<u>77.94</u>	38.96	57.30	64.96	82.10
	F1	20	21.33	31.19	21.27	39.95	<u>70.33</u>	37.00	33.53	40.13	74.63
		40	26.90	37.12	33.53	43.27	<u>75.40</u>	36.94	45.23	56.25	79.36
		60	27.28	37.77	40.17	46.58	<u>77.97</u>	37.35	56.49	65.73	81.81
CoraFull	ACC	20	29.33	17.78	53.38	56.68	41.68	<u>58.44</u>	54.50	47.74	58.90
		40	36.23	25.01	58.22	60.60	44.80	<u>62.98</u>	60.28	57.20	63.62
		60	40.60	29.65	59.84	62.00	46.68	<u>64.38</u>	61.58	60.18	65.36
	F1	20	28.05	18.24	47.59	52.48	37.15	<u>54.44</u>	50.44	45.07	54.74
		40	33.29	25.43	53.47	55.57	40.42	<u>58.30</u>	56.26	53.55	59.19
		60	37.95	30.87	54.15	56.24	43.22	<u>59.61</u>	57.26	56.40	61.32

4.2 节点分类

节点分类结果汇总于表 2 中，其中 L/C 表示每个类别中的有标签节点数量（the number of labeled nodes per class）。我们得到以下观察结果：

- 与所有基准方法相比，提出的 AM-GCN 模型在所有数据集和所有标签比例下通常都取得了最佳性能。
特别是在准确率（ACC）指标上，AM-GCN 在 BlogCatalog 数据集上取得了最高 8.59% 的相对提升，在 Flickr 数据集上取得了最高 8.63% 的相对提升。
实验结果证明了 AM-GCN 模型的有效性。
- AM-GCN 在所有数据集上均持续优于 GCN 和 kNN-GCN，
这表明 AM-GCN 中自适应融合机制（adaptive fusion mechanism）的有效性。
这是因为 AM-GCN 能够提取比单独使用 GCN 或 kNN-GCN 更多的有效信息。

- 通过与 GCN 和 kNN-GCN 的比较，我们可以发现拓扑图（topology graph）和特征图（feature graph）之间确实存在结构差异。
在传统拓扑图上执行 GCN 并不一定总能取得比基于特征图的方法更好的结果。
例如，在 BlogCatalog、Flickr 和 UAI2010 数据集中，特征图的表现优于拓扑图。
这进一步证明了在 GCN 中引入特征图的必要性。
- 此外，与 GCN 相比，AM-GCN 在具有更优特征图（kNN）的数据集上的提升更加明显，例如 UAI2010、BlogCatalog 和 Flickr。
这表明 AM-GCN 引入了一个更优、更适合的 kNN 图，用于在标签监督下进行特征传播（feature propagation）和节点表示学习（node representation learning）。

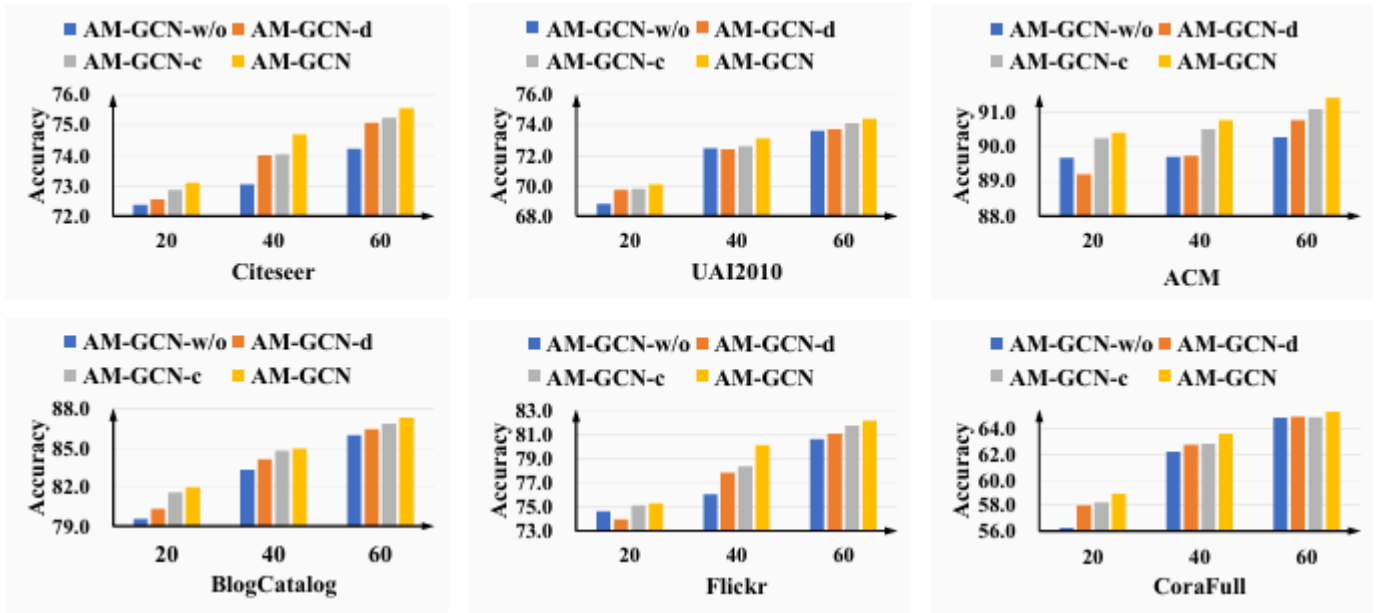


图 2：AM-GCN 及其变体（variants）在六个数据集上的实验结果（%）。

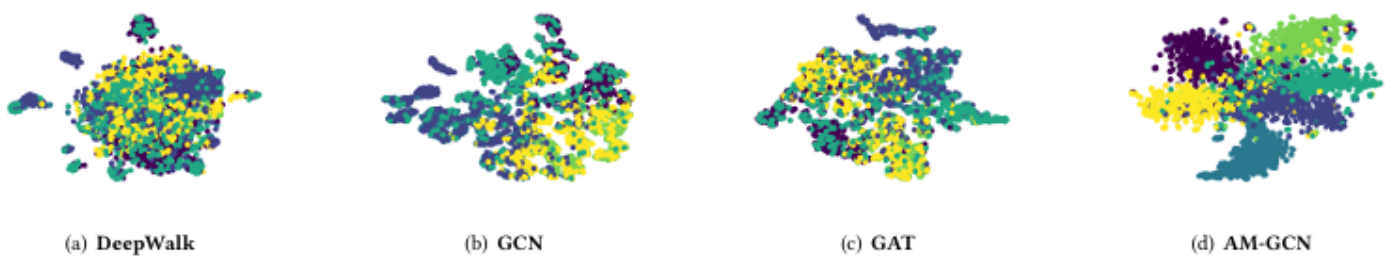


图 3：在 BlogCatalog 数据集上学习得到的节点嵌入（node embeddings）的可视化结果。

4.3 变体分析

在本节中，我们将 AM-GCN 与其三个变体在所有数据集上进行比较，以验证约束项的有效性。

- **AM-GCN-w/o**：不包含约束项 L_c 和 L_d 的 AM-GCN。
- **AM-GCN-c**：包含一致性约束 L_c 的 AM-GCN。
- **AM-GCN-d**：包含差异性约束 L_d 的 AM-GCN。

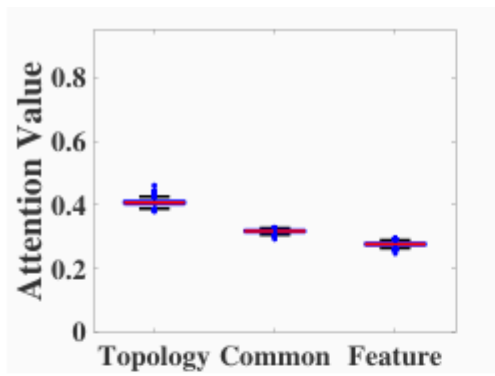
根据图 2 的实验结果，我们可以得到以下结论：（1）AM-GCN 的结果始终优于其他三个变体，说明同时使用两个约束项能够有效提升模型性能。（2）在所有数据集和所有标签比例下，AM-GCN-c 和 AM-GCN-d 的结果通常优于 AM-GCN-w/o，验证了一致性约束和差异性约束的有效性。（3）AM-GCN-c 通常优于 AM-GCN-d，说明在该框架中，一致性约束发挥了更加重要的作用。（4）通过比较图 2 和表 2 的结果可以发现，虽然 AM-GCN-w/o 没有使用任何约束项，但它仍然能够相对于基准方法取得非常有竞争力的性能，说明我们提出的框架具有较好的稳定性和竞争力。

4.4 可视化

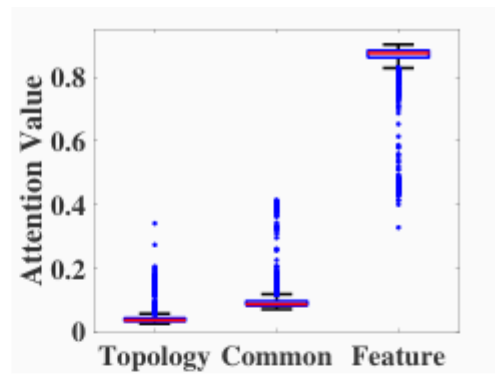
为了进行更加直观的比较，并进一步展示所提出模型的有效性，我们在 BlogCatalog 数据集上进行了可视化实验。我们使用 AM-GCN（或 GCN、GAT）最后一层 softmax 之前的输出嵌入（output embedding），并利用 t-SNE 对测试集学习得到的嵌入表示进行可视化。图 3 中 BlogCatalog 数据集的结果根据真实标签（real labels）进行着色。从图 3 可以发现 DeepWalk、GCN 和 GAT 的结果并不理想，因为不同标签的节点相互混合在一起。相比之下，AM-GCN 的可视化效果最好，其学习到的嵌入具有：更紧凑的结构（more compact structure），更高的类内相似性（higher intra-class similarity），不同类别之间更加清晰的边界（clearer distinct boundaries among different classes）。

4.5 注意力机制分析

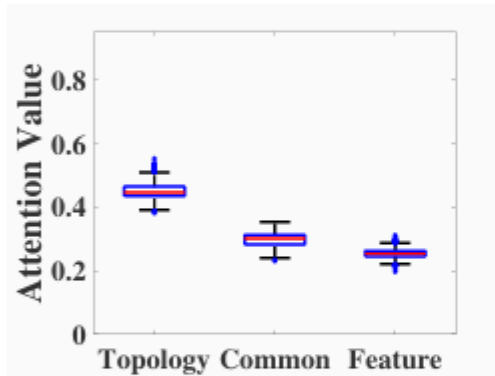
为了研究我们提出的模型学习到的注意力值（attention values）是否具有实际意义，我们分别分析了注意力分布（attention distribution）和注意力学习趋势（attention learning trend）。



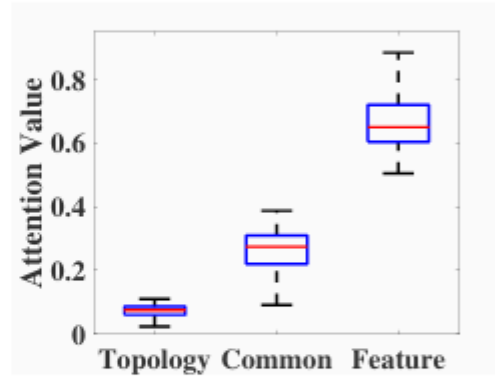
(a) Citeseer.



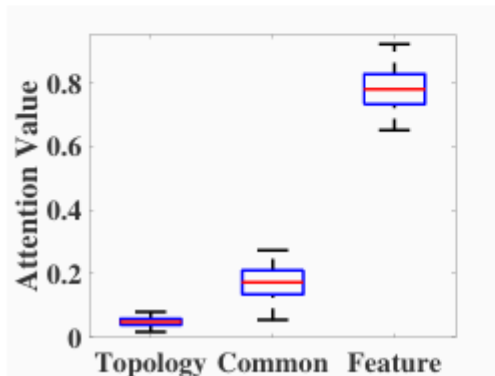
(b) UAI2010.



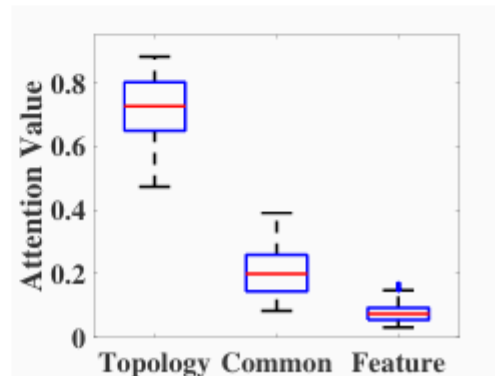
(c) ACM.



(d) BlogCatalog.

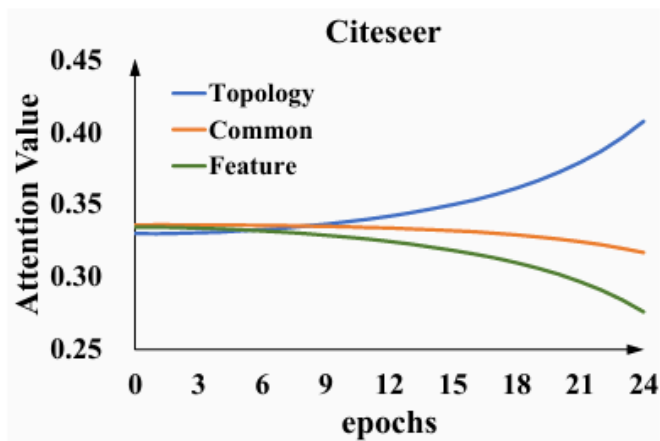


(e) Flickr.

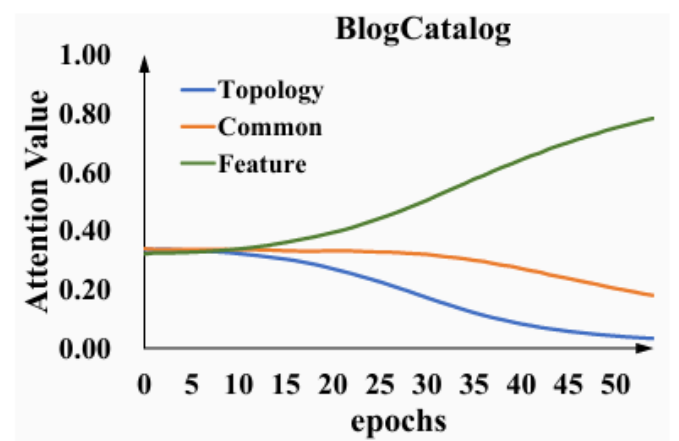


(f) CoraFull.

图 4：注意力分布分析。



(a) Citeseer



(b) BlogCatalog

图5：相对于训练轮的注意力变化趋势。

注意力分布分析

AM-GCN 学习了两种特定嵌入（specific embeddings）和一种公共嵌入（common embedding），每种嵌入都对应相应的注意力值。我们在所有数据集、20 个标签节点比例（20 label rate）的情况下进行了注意力分布分析，实验结果如图 4 所示。可以观察到对于 Citeseer、ACM 和 CoraFull 数据集，拓扑空间（topology space）中特定嵌入的注意力值大于特征空间（feature space）中的注意力值；公共嵌入（common embedding）的注意力值介于两者之间。这说明：拓扑空间中的信息可能比特征空间中的信息更加重要。为了验证这一点，我们可以观察表 2,在这些数据集中，GCN 的结果优于 kNN-GCN。相反，对于 UAI2010、BlogCatalog 和 Flickr 数据集,结合图 4 和表 2 可以发现,kNN-GCN 的性能优于 GCN；同时，特征空间中特定嵌入的注意力值也大于拓扑空间中的注意力值。总而言之，该实验说明,AM-GCN 能够根据不同信息的重要程度，自适应地分配更大的注意力权重给更重要的信息。

注意力趋势分析

我们进一步分析了训练过程中注意力值的变化趋势。这里以 Citeseer 和 BlogCatalog 数据集（20 个标签节点比例）为例，实验结果如图 5 所示：横轴（x-axis）表示训练轮数（epoch）；纵轴（y-axis）表示平均注意力值（average attention value）。更多实验结果见补充材料 A.4.1。在训练初始阶段：拓扑空间（Topology）,特征空间（Feature），公共空间（Common）三种嵌入对应的平均注意力值几乎相同。

随着训练轮数增加，三者的注意力值逐渐产生差异。例如，在 BlogCatalog 数据集中，拓扑空间的注意力值逐渐降低；特征空间的注意力值持续增加。这一现象与表 2 和图 4 的结论一致：即：kNN-GCN 使用的特征图性能优于 GCN 使用的拓扑图，说明在该数据集中，特征空间的信息比拓扑空间的信息更加重要。因此可以看出，AM-GCN 能够在训练过程中逐步学习不同嵌入表示的重要程度。

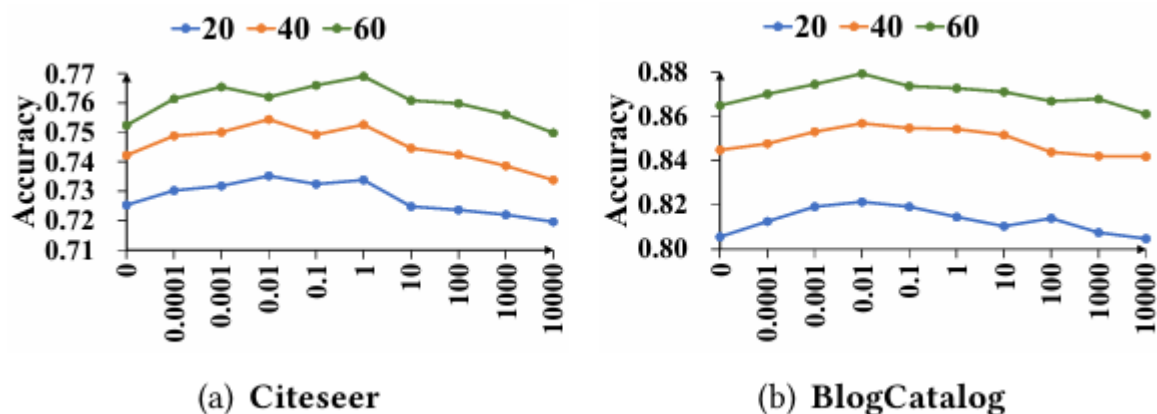
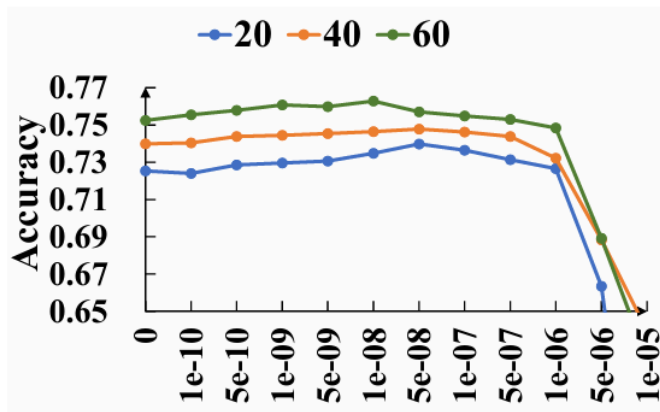
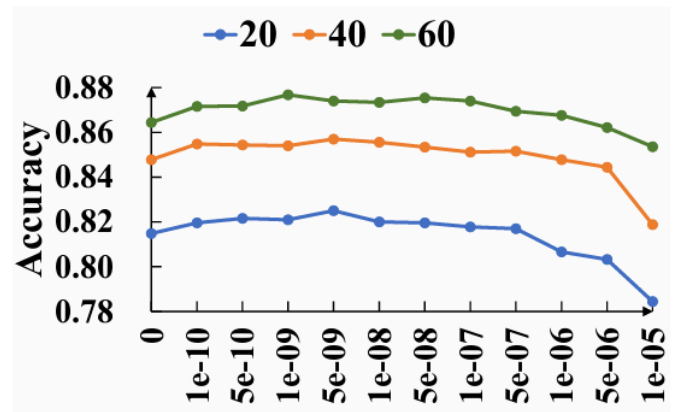


图6：参数(γ)的分析 (*Analysis of parameter(γ)*)。

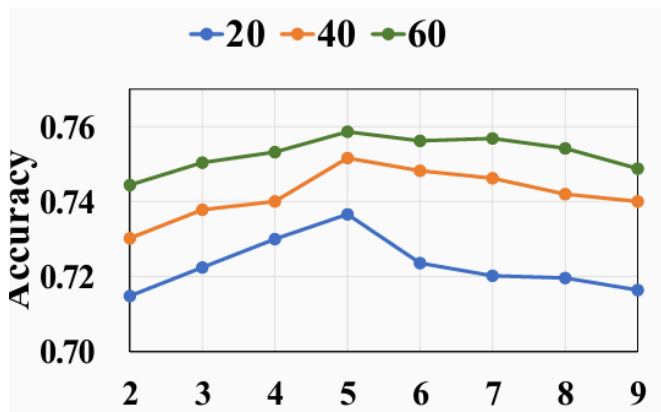


(a) Citeseer

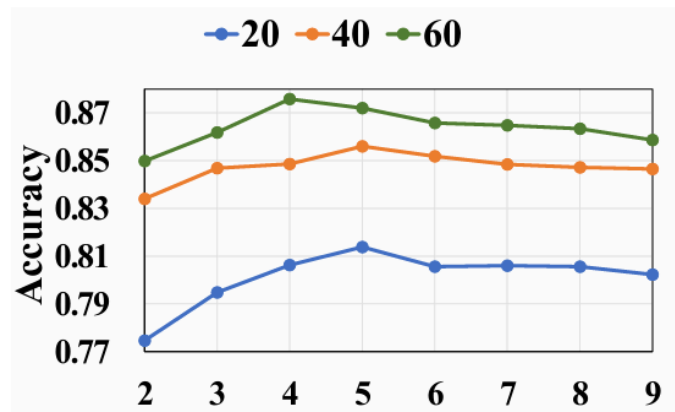


(b) BlogCatalog

图7: 参数 β 的分析 (*Analysis of parameter(β)*)。



(a) Citeseer



(b) BlogCatalog

图8: 参数 k 的分析 (*Analysis of parameter(k)*)。

4.6 参数研究

在本节中，我们研究模型参数在 Citeseer 和 BlogCatalog 数据集上的敏感性。更多实验结果见补充材料 A.4.2。

一致性约束系数 γ 分析

我们测试了公式 (18) 中一致性约束权重 γ 对模型性能的影响，并将 γ 的取值范围设置为 $0 \sim 10000$ ，实验结果如图 6 所示。随着一致性约束系数的增加，模型性能首先提升，随后开始缓慢下降。总体而言，当： $\gamma \in [1e^{-4}, 1e^4]$ 时，AM-GCN 在所有数据集上都表现出较好的稳定性。我们还可以观察到，不同标签比例（20、40、60 个标签节点）的曲线呈现出相似的变化趋势。

差异性约束系数 β 分析

随后，我们测试公式 (18) 中差异性约束权重 β 的影响，并将 β 的取值范围设置为： $0 \sim 1e^{-5}$ 实验结果如图 7 所示。类似地，随着 β 的增加，模型性能首先提升。但是对于 Citeseer 数据集，如图 7(a) 所示，当： $\beta > 1e^{-6}$

时，模型性能会快速下降；对于 BlogCatalog 数据集，模型性能相对更加稳定。

k 近邻图参数k分析

为了研究 kNN 图中最大邻居数量 k 对模型性能的影响，我们测试了不同 k 值下 AM-GCN 的性能，其中： $k = 2 \sim 10$ 实验结果如图 8 所示。对于 Citeseer 和 BlogCatalog 数据集：随着 k 增加，准确率首先提升，然后开始下降。可能原因是：当图结构变得更加稠密（denser）时，特征更容易被平滑（feature smoothing）；较大的 k 值可能会引入更多噪声边（noisy edges）。因此，过大的 k 值可能会降低模型性能。

5. 相关工作

近年来，图卷积网络（Graph Convolutional Network, GCN）模型 [4, 9, 17, 23, 33, 35] 得到了广泛研究。例如：文献 [3] 首次通过图拉普拉斯矩阵（graph Laplacian）在傅里叶域（Fourier domain）中设计了图卷积操作。文献 [5] 进一步利用图拉普拉斯矩阵的切比雪夫展开（Chebyshev expansion）来提高计算效率。文献 [14] 简化了卷积操作，并提出仅聚合一阶邻居（one-hop neighbors）的节点特征。GAT [27] 引入注意力机制（attention mechanism），利用学习得到的权重对节点特征进行聚合。GraphSAGE [11] 提出通过对节点局部邻域进行采样（sample）并聚合（aggregate）特征，采用 mean/max/LSTM pooling 方法生成节点表示。DEMO-Net [31] 设计了一种基于节点度数感知（degree-aware）的特征聚合过程。MixHop [1] 在每一层网络中同时聚合来自一阶邻居和高阶邻居（first-order and higher-order neighbors）的特征信息。目前大多数 GCN 方法主要关注于融合网络拓扑结构（network topology）和节点特征（node features），从而学习用于分类任务的节点嵌入表示（node embedding）。此外，近年来也出现了一些研究工作用于分析 GCN 中的信息融合机制。例如：文献 [15] 表明，GCN 实际上是在节点特征上执行拉普拉斯平滑（Laplacian smoothing）。文献 [20] 和 [30] 证明，拓扑结构在节点特征上起到了低通滤波（low-pass filtering）的作用。关于 GCN 的更多研究工作，可以参考详细的综述文章 [32, 38]。然而，GCN 是否能够自适应地从节点特征和拓扑结构中提取与分类任务相关的信息，目前仍然尚不明确。

6. 结论

在本文中，我们重新思考了 GCN 中网络拓扑结构（network topology）与节点特征（node features）的融合机制，并发现这种融合方式距离最优状态仍存在较大差距。受到这一基本问题的启发，我们研究了如何从拓扑结构和节点特征中自适应地学习与任务最相关的信息，并将这些信息充分融合用于分类任务。我们提出了一种多通道模型 AM-GCN，该模型能够在融合拓扑信息和节点特征信息时，学习合适的重要性权重（importance weights）。大量实验结果表明，在真实世界数据集上，AM-GCN 相比当前最先进的方法（state-of-the-art models）取得了更优越的性能。

A 补充材料

A.1 实验设置

所有实验均在以下环境下完成：

- **操作系统 (Operating System)** : CentOS Linux release 7.6.1810
- **CPU**: Intel(R) Xeon(R) CPU E5-2620 v4 @ 2.10GHz
- **GPU**: GeForce GTX 1080 Ti
- **软件版本 (Software versions)** : Python 3.7, PyTorch 1.1.0, NumPy 1.16.2, SciPy 1.3.1, NetworkX 2.4, scikit-learn 0.21.3

A.2 基准方法和数据集

基准方法 (Baselines) 的公开实现代码可以在以下网址中找到:

- **DeepWalk, LINE**: <https://github.com/thunlp/OpenNE>
- **Chebyshev**: <https://github.com/tkipf/gcn>
- **PyTorch 版本 GCN**: <https://github.com/tkipf/pygcn>
- **PyTorch 版本 GAT**: <https://github.com/Diego999/pyGAT/>
- **DEMO-Net**: <https://github.com/jwu4sml/DEMO-Net>
- **MixHop**: <https://github.com/samihaija/mixhop>

本文使用的数据集可以通过以下网址获取:

- **Citeseer**: <https://github.com/tkipf/pygcn>
- **UA12010**: <http://linqs.umi.acs.umd.edu/projects//projects/lbc/index.html>
- **ACM**: <https://github.com/Jhy1993/HAN>
- **BlogCatalog**: <https://github.com/mengzaiqiao/CAN>
- **Flickr**: <https://github.com/mengzaiqiao/CAN>
- **CoraFull**: <https://github.com/abojchevski/graph2gauss/>

A.3 实现细节

AM-GCN 的代码实现基于 PyTorch 版本 1 的 Graph Convolutional Networks (GCN) 代码框架。

为了保证所提出模型的可复现性, 我们在表 3 中列出了模型中使用的参数取值。

A.4 额外实验结果

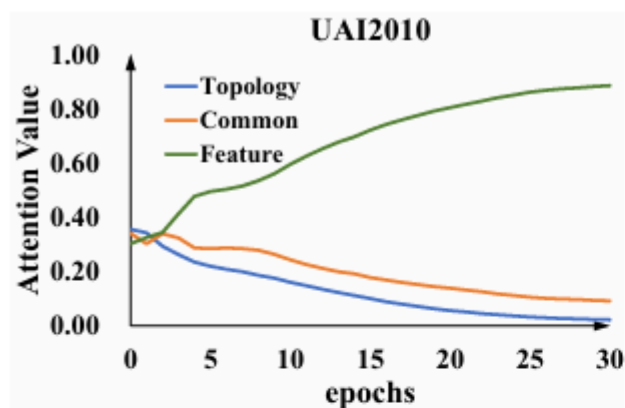
在本节中, 我们提供额外的实验结果, 包括:

- 其他四个数据集上的注意力趋势分析 (analysis of attention trends) ;
- UA12010 和 Flickr 数据集上的参数研究 (parameter study) 。

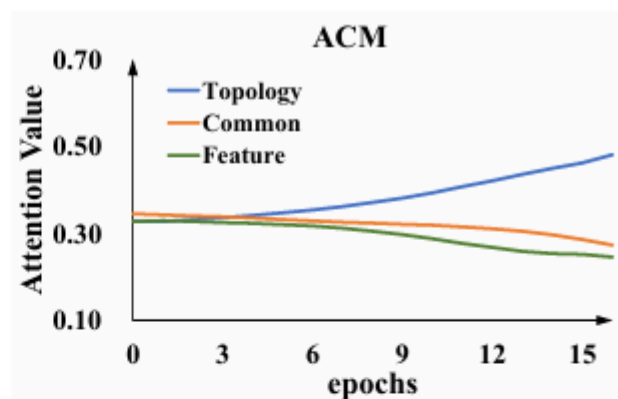
A.4.1 注意力趋势分析

根据第 4.5 节中的实验设置, 我们在图 9 中给出了其他四个数据集上的注意力趋势额外分析结果。

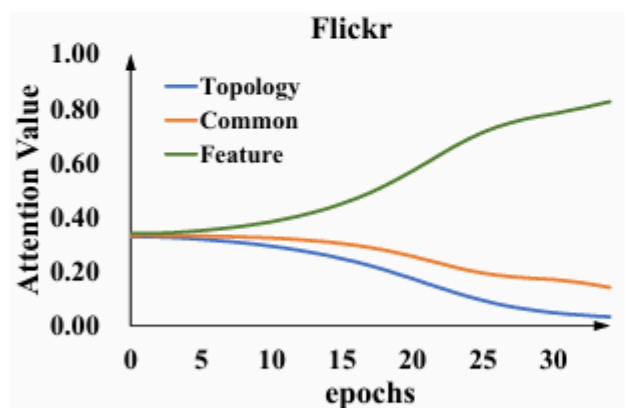
注意力值的变化过程与图 5 中的实验结果保持一致。此外，最终学习得到的注意力值与图 4 中对应的注意力分布结果相一致。这进一步验证了注意力机制（attention mechanism）的有效性。



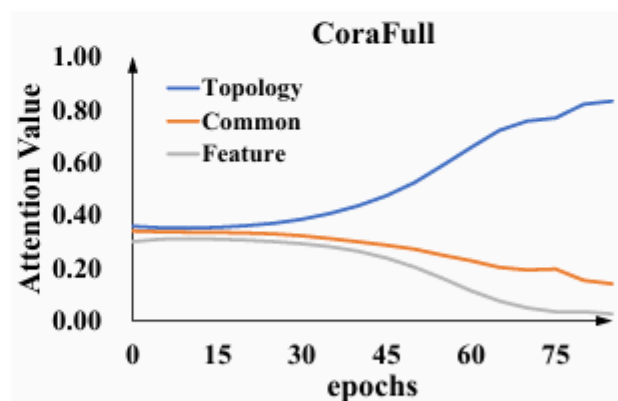
(a) UAI2010



(b) ACM

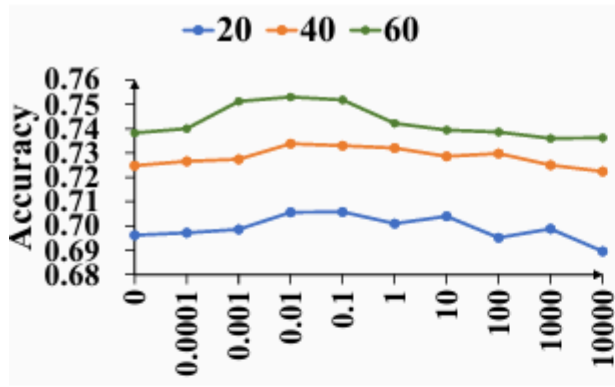


(c) Flickr

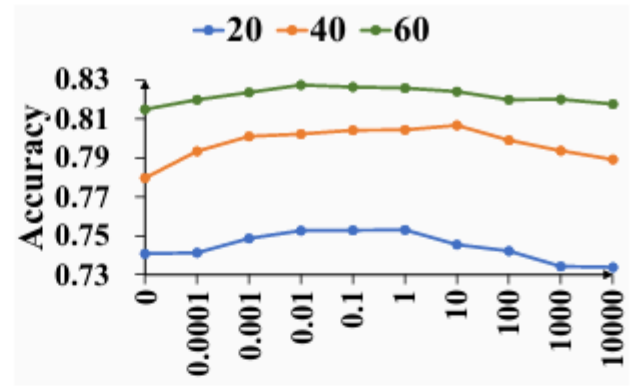


(d) CoraFull

Figure 9: Changing trends on another four datasets.

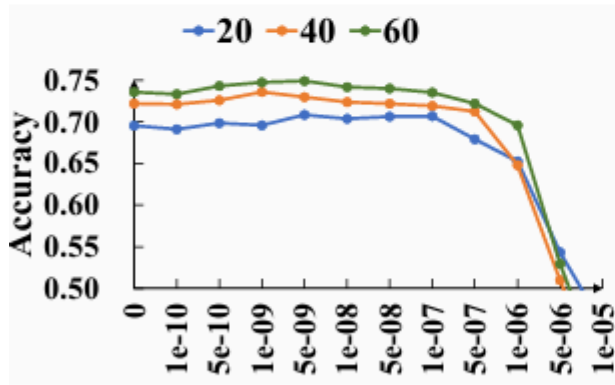


(a) UAI2010

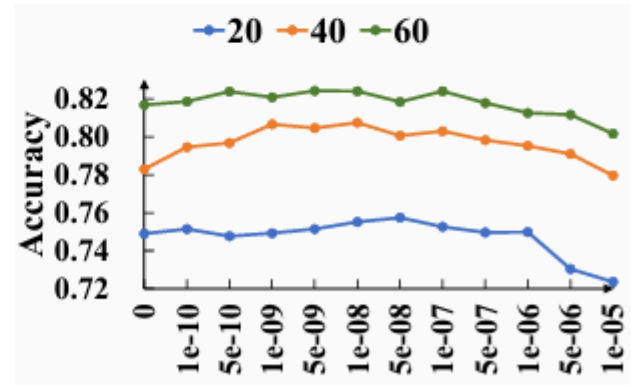


(b) Flickr

Figure 10: Analysis of parameter γ .

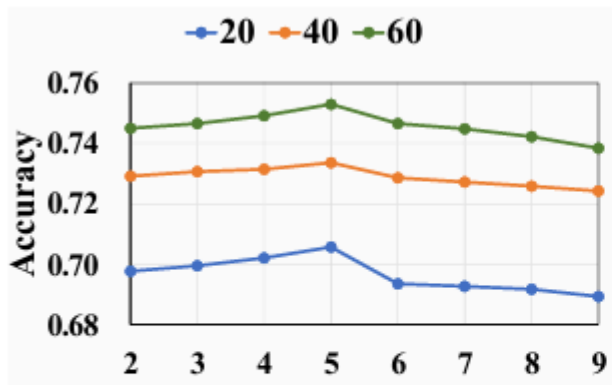


(a) UAI2010

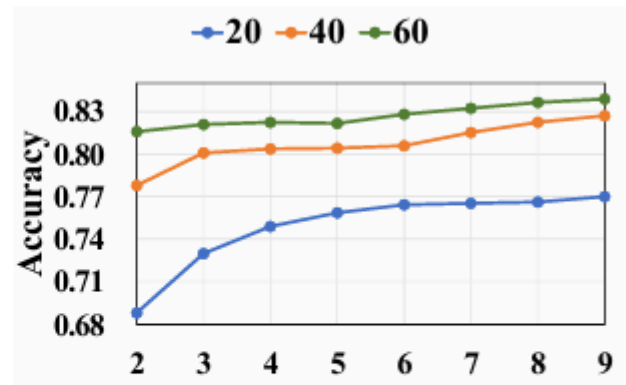


(b) Flickr

Figure 11: Analysis of parameter β .



(a) UAI2010



(b) Flickr

Figure 12: Analysis of parameter k .

Table 3: Model Hyperparameters.

Datasets	L/C	nhid1	nhid2	dropout	lr	weight-decay	epoch _{max}	k	γ	β
Citeseer	20	768	256	0.5	0.0005	5e-3	25	7	0.001	5e-10
	40	768	128	0.5	0.0005	5e-3	25	7	0.001	5e-8
	60	768	128	0.5	0.0005	5e-3	25	7	0.001	5e-8
UAI2010	20	512	128	0.5	0.0005	5e-4	50	5	0.001	1e-9
	40	512	128	0.5	0.0005	5e-4	70	5	0.01	1e-9
	60	512	128	0.5	0.0005	1e-5	70	5	0.01	1e-9
ACM	20	768	256	0.5	0.0005	5e-4	20	5	0.001	1e-8
	40	768	256	0.5	0.0005	5e-4	20	5	0.001	1e-8
	60	768	256	0.5	0.0001	6e-4	30	5	0.001	1e-8
BlogCatalog	20	512	128	0.5	0.0002	1e-5	55	5	0.001	5e-8
	40	512	128	0.5	0.0005	5e-4	40	5	0.001	5e-8
	60	512	128	0.5	0.0005	8e-4	50	5	0.01	5e-8
Flickr	20	512	128	0.5	0.0003	5e-4	60	5	0.01	1e-10
	40	512	128	0.5	0.0005	1e-5	40	5	0.01	1e-10
	60	512	128	0.5	0.0005	5e-4	40	5	0.01	1e-10
CoraFull	20	512	32	0.5	0.001	5e-4	300	6	0.0001	1e-10
	40	512	32	0.5	0.001	5e-4	300	6	0.00001	1e-10
	60	512	32	0.5	0.001	5e-4	300	6	0.0001	1e-10

A.4.2 参数研究

为了进一步验证参数 γ 和 β 的稳定性以及适用性，我们在 UAI2010 和 Flickr 数据集上展示了相应的实验结果，如图 10 和图 11 所示。

结合第 4.6 节中的实验结果，我们可以发现，一致性约束（consistency constraint）和差异性约束（disparity constraint）在较大的参数范围内都能够保持稳定的性能；当差异性约束系数 β 超过某个合适的范围后，模型性能可能会下降。在图 12 中，我们测试了 k 近邻图（ k -nearest neighbor graph）中参数 $\times k$ 对 UAI2010 和 Flickr 数据集的影响。实验结果表明：对于 UAI2010 数据集，当 k 从 2 增加到 10 时，模型性能首先提升，然后开始下降。对于 Flickr 数据集，较大的 k 值可能为特征图（feature graph）引入更加丰富的结构信息，从而进一步提升 AM-GCN 的性能。